

Step-by-step Reduction Procedure

Last updated: 08/04/2025

Note: Be cautious copying and pasting from this page! Mac terminals often misread space characters, and cannot process the ' character, only the ' character. If your code isn't running and you can't figure out why, try re-typing the entire command manually into your terminal.

Downloading data:

The archival process with Gemini has been updated recently. Eventually it will all be automated, but at the moment only public programs are automatic (calibration files and internal programs). For now, these four steps need to be completed, in order, per date, from mxcrunch in /data10/GSA_download/.

```
curl https://archive.gemini.edu/download/canonical/MAR00N-X/20250322 -o  
GSA_archive.tar
```

```
tar -xvf GSA_archive.tar
```

```
ls -l N20250322M* > 20250322.lis
```

```
python ~/Repos/maroonx_reduce/split_MEF_MAR00NX.py -l 20250322.lis
```

Here was the older routine. It should be obsolete now, but just for reference:

From MaroonX_spectra on mxcrunch, list available folders and their IDs:

```
gdrive files list --drive 0AClc1049ft3YUk9PVA --max 50
```

Download a folder by calling its ID:

```
gdrive files download --recursive 19WUGTkPhe5akXb7aba2QxPsiWT0qxRog
```

Darks:

This process was recently-ish updated. Since December 2023, we've been using a set of 'real' masterflats to build coefficients that have been applied to create synthetic darks for a given exposure time and ND filter. This all needs to be done on mxred.

To create masterdarks from a set of darks, you can first run (from /home/maroonx/Repos/maroonx_reduce on mxred) `'PYTHONPATH=${PWD} python reduce/recipes/make_master_darks.py -dd '/data10/MaroonX_spectra/' -d '20231122' -t 600'`, adjusting for date and exposure time.

To create the coefficients, you would make a list of input 'real' masterdarks and create a '202311xx/darks' subdirectory, then run (per arm) `'PYTHONPATH=${PWD} python reduce/recipes/make_coeffs_from_masterdarks.py -l '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/masterdarks_202311xx_red.lis' --overwrite -od '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/202311xx/darks/' -cr 'masterdarks_202311xx_red.npz'`', where -l is the input list you just made. This step takes around an hour, and doesn't give a lot of outputs. This is done on mxred in maroonx_reduce.

After creating the coefficients, you would only need to run `'PYTHONPATH=${PWD} python reduce/recipes/make_master_darks_from_coeffs.py -dd '/data10/MaroonX_spectra/' -od '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/'`

202410xx/darks/' -cr 'masterdarks_coeffs_202410xx_red.npz' -cb 'masterdarks_coeffs_202410xx_blue.npz' -d '202312??', and the code will look for all exposure time and ND filter combinations from the selected dates. It'll use the previously calculated coefficients to create masterdarks based on those criteria. This is also done on mxred in maroonx_reduce. Since December, I have only followed the third step to create darks.

To run flux extractions on these darks (typically for QC purposes), you'd run: `PYTHONPATH=${PWD} python reduce/extraction.py -s '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/202410xx/darks/202410xx_masterdark_mean_DDDDE_r_0300.fits' -f '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/202409xx/flats/20240916T19_masterflat_backgrounds_subtracted_FFFFF_r_0002.hdf' -c '/data/MaroonX_spectra_reduced/Maroonx_configfiles/202005xx/config_r.hdf' -o '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/202410xx/darks_extracted/202410xx_masterdark_mean_DDDDE_r_0300.hdf' -fibers 5 -sd 5 -flat_fibers '2-5' -over True -dbg 0'`

Special case (LFC darks):

We prefer to make one dark per day (AM + PM combined). We don't need to make synthetic darks since we take these daily. The LFC darks are a little tricky since the colors have different exposure times. I use 'ls >' to make an input list per month of LFC darks (I keep these lists in Maroonx_masterframes). Currently the blue exposure times are 10s, and red are 30s, but be cautious as that might change.

It's wildly tedious, but I do this per day: `ls /data10/MaroonX_spectra/20250501/*DDDDDE_r_0004.fits > /data10/MaroonX_spectra_reduced/Maroonx_masterframes/202505_LFC_rawdarks_red.lis` use and then `PYTHONPATH=${PWD} python reduce/recipes/make_master_darks.py -dd '/data10/MaroonX_spectra/' -l '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/202505_LFC_rawdarks_red.lis' -d '20250501'` per day and color.

Flats:

This procedure follows four steps, and only needs to be done roughly once per month.

First, you would create combined masterflats of each image type (FDDDF and DFFFD) by running `PYTHONPATH=${PWD} python reduce/recipes/make_master_flats.py -dd /data10/MaroonX_spectra/20240401/'`, needing only to specify the date. If any filenames follow an abnormal format, you can specify the input filename format. For example, recently we've been taking DDDDF frames rather than FDDDF frames, due to difficulty illuminating the sky fiber. We would specify the input file format as, `--filename_FDDDF_b *DDDDF_b*`, and the same for the red channel.

Next, you would run flux extractions on the illuminated orders for each flat type.

`PYTHONPATH=${PWD} python reduce/extraction.py -s '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/202404xx/flats/20240401T00_masterflat_DFFFD_r_0002.fits' -f '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/202404xx/flats/20240401T00_masterflat_DFFFD_r_0002.fits' -c '/data/MaroonX_spectra_reduced/Maroonx_configfiles/202005xx/config_r.hdf' -o '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/202404xx/flats/' -dbg 1 -over True -fibers '2 3 4' -flat_fibers '2 3 4'`. This produces hdf files of each flat type.

These will be put in a subfolder and need to be moved into the parent directory before proceeding.

Next, you would run a background subtraction, specifying the data directory `'PYTHONPATH=${PWD} python reduce/recipes/backgroundfit.py -dd /data10/MaroonX_spectra_reduced/Maroonx_masterframes/202404xx/flats/'`. Like the first step, you can specify the filename format if it's abnormal. Side-note: I've had issues with the code calling the masterflat_median .fits rather than just the masterflat .fits — specifying the file inputs as `*masterflat_DFFFD_b*`, or the like, should take care of this bug.

The final step is to run flux extractions on the background subtracted masterflats, following the same form as the second step. `'PYTHONPATH=${PWD} python reduce/extraction.py -s '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/202404xx/flats/20240401T00_masterflat_backgroundsubtracted_FFFFF_r_0002.fits' -f '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/202404xx/flats/20240401T00_masterflat_backgroundsubtracted_FFFFF_r_0002.fits' -c '/data10/MaroonX_spectra_reduced/Maroonx_configfiles/202005xx/config_r.hdf' -o '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/202404xx/flats/' -dbg 1 -over True -fibers '2 3 4 5' -flat_fibers '2 3 4 5''`. These will also be put in a subfolder and need to be moved into the parent directory before proceeding.

To create an hd5 pandas export, you'd run: `'PYTHONPATH=${PWD} python analyze/recipes/batch_spec_to_pandas.py -dd /data10/MaroonX_spectra_reduced/Maroonx_masterframes/202409xx/flats/ -o FFFFF -d '20240916T19' --file '20240916T19_masterflat_backgroundsubtracted_FFFFF_b_0007.hdf''`

Science flux extractions:

With the new dark procedures, this can be run on a range of dates. It is also quite easy to overload the server's memory if enough files are run through at once, so usually no more than 50 should be run through at a time. This command is the typical format, `'PYTHONPATH=${PWD} python reduce/batch_extraction.py -c 'b' -fb '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/202411xx/flats/20241114T19_masterflat_backgroundsubtracted_FFFFF_b_0007.hdf' -dd '/data10/MaroonX_spectra/' -od '/data10/MaroonX_spectra_reduced/' -e '0300' -sl 5 -t S000E -d '202411*' -dkd 202410xx'`, specifying the date, exposure time, color, and flat file to be used. This step must be run per exposure time, to ensure that the correct corresponding dark is called.

Etalon flux extractions:

Similar format as the science flux extractions, but specifying a different filetype. This would need to be run for each DEEEE/DLLL? r/b filetype and color: `'PYTHONPATH=${PWD} python reduce/batch_extraction.py -c 'b' -fb '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/202409xx/flats/20240916T19_masterflat_backgroundsubtracted_FFFFF_b_0007.hdf' -dd '/data10/MaroonX_spectra/' -od '/data10/MaroonX_spectra_reduced/' -e '*' -t 'DLLL?' -d '202311*' -sd 'all''`. This step typically doesn't take too much memory, and can be run on a ton of data at a time.

Fitting etalon frames:

This step fits the etalon lines in the etalon frames: `'PYTHONPATH=${PWD} python reduce/batch_extract_etalon_positions.py -dd '/data10/MaroonX_spectra_reduced/' -fb '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/202411xx/flats/20241114T19_masterflat_backgroundsubtracted_FFFFF_b_0007.hdf' -c 'b' -e '????' -t 'DEEEE' -d '202311*' -fs 2-5'`. This can be run on large ranges of dates, and is run on DEEEE/DLLL? r/b filetypes and colors.

Fitting science frames:

Follows the same format as fitting etalon frames, only differing the input filetype and fibers to fit: `'PYTHONPATH=${PWD} python reduce/batch_extract_etalon_positions.py -dd '/data10/MaroonX_spectra_reduced/' -fb '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/202411xx/flats/20241114T19_masterflat_backgroundsubtracted_FFFFF_b_0007.hdf' -c 'b' -d '2025010?' -t S000E -e '*' -fs 5'`.

After this step is good point to run `'fgrep */*.log 'ERROR''` to look for any fitting issues. Most common science errors is a peak max/min mismatch. This is usually caused by image artifacts or cosmic rays that weren't handled properly. This can be fixed by providing an etalon reference, by adding the argument `'-g '/data10/MaroonX_spectra_reduced/20250101/20250101T000000_DEEEE_b_0030.hdf''`.

Wavelength and drift solutions:

Only run on DEEEE frames, per color: `'PYTHONPATH=${PWD} python analyze/recipes/batch_etalon_spline_wls.py -d '202311??' -c 'b' -m True -dd /data10/MaroonX_spectra_reduced/ -o 'DEEEE''`.

Transferring solutions:

Transfers the etalon WLS to science frames, run per color: `'PYTHONPATH=${PWD} python analyze/recipes/batch_science_spline_wls_dynamic.py -d '202311??' -c 'r' -m True -dd /data10/MaroonX_spectra_reduced/ -o 'S000E''`. The most common error I find here is 'date/time delta not found', which means that there wasn't an etalon solution available. Make sure the etalon wavelength solution are completed before this step.

Combine to fiber 6:

Combines fibers 2-4 into an artificial fiber 6: `'PYTHONPATH=${PWD} python analyze/recipes/combine_science_fibers.py -dd '/data10/MaroonX_spectra_reduced/' -d '202311??' --max_clip 20'`.

Barycentric velocity measurements:

Corrects for barycentric radial velocities. This step relies on flux meter readings (which are uploaded automatically with the raw data as pickle files): `'PYTHONPATH=${PWD} python analyze/recipes/batch_barycor.py -dd '/data10/MaroonX_spectra_reduced/' -d '202311??' -n 'GL 15 A''`.

If the target name is not SIMBAD-recognizable, and the target is in SIMBAD under a different name, then you would add an additional parameter `'--sn'` with the SIMBAD-recognizable name. If the target is not in SIMBAD at all, you would instead add the parameter and parameter value `'--use_coord TRUE'`.

The code automatically finds a zeropoints for the PC and FRD counts. This gets finicky when there's three or more observations of the same target, and the middle observation(s) don't have the zeropoints appear in or around the exposure. In these cases, you would set the zeropoints manually under the parameters `'--zp_pc -zp_frd'`.

Use `'xdg-open '20250101/20250101T000000_S000E_r_exposuremeter.pdf''` to check any files where you're wary about the PC/FRD values.

For pre-2025 data, the exposuremeter data was not uploaded with the FITS files like they are now. Instead, it would be called with the BC script as an extra argument, as `'-p '/data10/MaroonX_spectra_reduced/Maroonx_masterframes/202311xx/expmeter/expmeter112023.pkl''`. This is still necessary if you're running this step on older data.

Pandas packaging:

For delivering data to external PI's, the red and blue hdf files for each exposure are combined into a single hd5 file, using the code: `'PYTHONPATH=${PWD} python analyze/recipes/batch_spec_to_pandas.py -dd /data10/MaroonX_spectra_reduced/ -d '202311??' -t '*'`.

Running SERVAL:

In mxred, go to `/home/maroonx/serval`, and create an input list of files like so, per arm: `'hdf_header.py -dd '/data10/MaroonX_spectra_reduced/2025*/' -f '2025*S000E*_b_????.hdf' -ff "TARGETNAME=GL 486" > GL486_2025A_blue.lis'`

Specify the date ranges as needed.

Then, go to `/home/maroonx/Repos/serval3_maroonx/src`, and activate the conda environment by running `'conda activate serval3_maroonx'`.

Next, run the following command, per arm:

```
'python maroonxserval.py --instrument MAROONXredcoadd --inputlist /home/maroonx/serval/GL486_2025A_red.lis --outputdir /home/maroonx/serval3/GL486_2025A/ --K_upper_limit 3000 --snrmax 10000 'GL 486''
```

The output will be saved to `/home/maroonx/serval3`. Be cautious not to overwrite any existing files.

One common error is that the target info was not found, in which case, you'll need to create a target info list. We keep these in /home/maroonx/Repos/serval3_maroonx/targets. You can create a target file by copying the format of another target file, and filling in the information to the best of your ability. Once SERVAL has finished running, check the stellar activity plots in /home/maroonx/serval3/target_date/target/MAROONXredcoadd/plots_stellar_activity/ to make sure that the H-alpha line and other indicators are correctly identified. If they're not, then change the target's systemic RV in the target file, and re-run SERVAL until the indicators are correctly lined up.

Another common error is that one of the input files is missing 'JD UTC' information. In that case, something likely went wrong with the BC corrections. Make sure the BC corrections are all correct for your target. There's a similar error and solution with 'fiber6'.

SERVAL results should be reviewed for very high SNR cases (> 1000). 'MAROONXred/blue' or 'MAROONXred/bluefiberX' might be a better option.

SERVAL is nearly useless on very low SNR cases (< 20). LBL or another alternative should be recommended to the PI.

Packaging up data:

FITS files (from /data10/MaroonX_spectra/):

```
dfits 2025*/S000E*.fits | fitsort 'HIERARCH MAROONX TELESCOPE  
PROGRAMID' | fgrep -i '25A-Q-221' > GN-2025A-Q-221_inlist_fits.txt
```

or swap out 'PROGRAMID' for 'TARGETNAME'

```
trim_dfits.sh GN-2025A-Q-221_inlist_fits.txt
```

```
tar -cvf GN-2025A-Q-221_FITS.tar -T GN-2025A-Q-221_inlist_fits.txt
```

```
gzip GN-2025A-Q-221_FITS.tar
```

QC files (from /data10/MaroonX_spectra_reduced/):

```
cp /data10/MaroonX_spectra/GN-2025A-Q-221_inlist_fits.txt .
```

```
convert_fits_to_logs.sh GN-2025A-Q-221_inlist_fits.txt
```

```
tar -cvf GN-2025A-Q-221_QC.tar -T GN-2025A-Q-221_inlist_fits.txt.logs
```

```
gzip GN-2025A-Q-221_QC.tar
```

HD5 files (from /data10/MaroonX_spectra_reduced/):

```
hdf_header.py -d '/data10/MaroonX_spectra_reduced/2025*' -f  
'2025*S000E_b_????.hdf' -ff 'PROGRAMID=GN-2025A-Q-221' > GN-2025A-  
Q-221_inlist_hd5.txt
```

or swap out 'PROGRAMID' for 'TARGETNAME'

```
convert_serval_to_hd5.sh GN-2025A-Q-221_inlist_hd5.txt
```

```
tar -cvf GN-2025A-Q-221_HD5.tar -T GN-2025A-Q-221_inlist_hd5.txt
gzip GN-2025A-Q-221_HD5.tar
```

If the tarring fails, check that the hd5 timestamps listed match the actual hd5 files.

These last two are not standard for every program. SERVAL is only used for normal RV programs, not chemical tagging programs or extremely low-signal observations. NumPy exports are typically only for single-night time-series observations.

SERVAL (from /home/maroonx/serval3/):

```
tar -cvf GN-2025A-Q-221_SERVAL.tar GL486_2025A (skip '-T')
gzip GN-2025A-Q-221_SERVAL.tar
```

NumPy exports (from /home/maroonx/Repos/maroonx_reduce/others/):

Copy a recent script and rename for your target and date. Use 'nano' to edit the file. Scroll down, and you should see this section.

```
if __name__ == "__main__":
    #filelist_red = np.genfromtxt('/home/maroonx/serval/WASP189_20220602_red.lis', dtype=str)
    #filelist_blue = np.genfromtxt('/home/maroonx/serval/WASP189_20220602_blue.lis', dtype=str)

    filelist_red = sorted(get_files("20230624*S000E_r_0135.hdf", "/data9/MaroonX_spectra_reduced/"))
    filelist_blue = sorted(get_files("20230624*S000E_b_0175.hdf", "/data9/MaroonX_spectra_reduced/"))

    flux_red, wavelengths_red, f_var_red, airmass, time, brv_red = spec_to_numpy(filelist_red, 67, 95, l=4036, meta=True, exptime=135)
    flux_blue, wavelengths_blue, f_var_blue, _, _, brv_blue = spec_to_numpy(filelist_blue, 92, 125, l=3954, meta=True, exptime=175)

    np.savez_compressed('/data9/MaroonX_spectra_reduced/MAR00N-X_KELT16_20230624',
        flux_red=flux_red,
        flux_blue=flux_blue,
        f_var_red=f_var_red,
        f_var_blue=f_var_blue,
        wavelengths_red=wavelengths_red,
        wavelengths_blue=wavelengths_blue,
        airmass=airmass,
        time=time,
        brv_red = brv_red,
        brv_blue = brv_blue)

    print()
```

The uncommented variables filelist_red and filelist_blue grab the desired files by sorting through date and exposure time. Adjust these parameters as needed. If your target has the same exposure time and date as another target (i.e., you have a time series of 300s exposures of WASP-107, but Barnard Star was observed with 300s just a few hours earlier), then you'd want to use the commented variables. For that case, you'd need to create an input list of reduced HDF files to pack.

In the next section, where we define flux, wavelength, f_var, etc., correct the exposure time at the end of the line to match your observations.

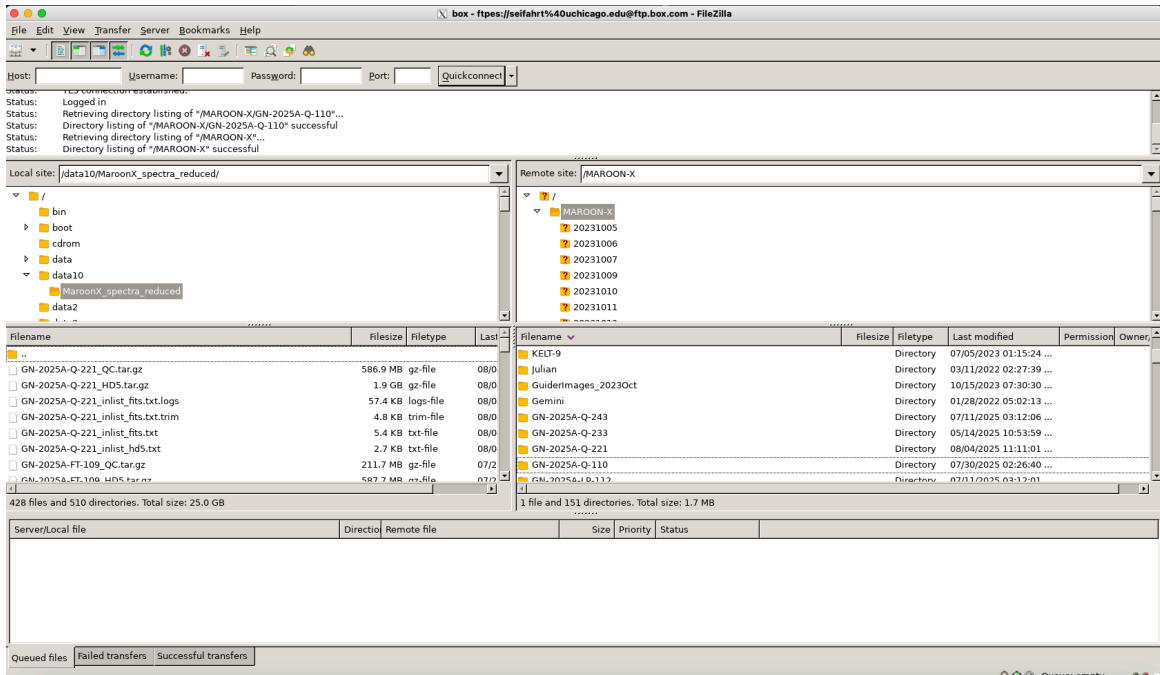
In the final section, rename the output file to match your target name and date. Save this script and exit when done.

After this, cd back to /home/maroonx/Repos/maroonx_reduce/ and run:

```
python others/KELT-11b_spec_to_numpy_20240419.py
```

Uploading to Box (from anywhere):

Log into Box Drive to create a folder for a given program. Go back to your terminal and use command **'filezilla'** to open our file transfer software. Press the connect button in the top left corner. You should see something like this now.



The left side navigates our files on our server, and the right side navigates the files on Box. On the right side, open the program folder you just created on Box. Find the packaged files you want to upload, and double-click them. They should show up in the empty box at the bottom with an upload progress bar.

You'll also need to upload the hd5 version of the flat used in reduction, as well as our **'plot_pandas_spec...'** Python script, customized to their program and observations.